

Detecting Non-Interference Violations in TCP State Machine Slices for Off-Path Side-Channel Attacks

Walid Ait Amrou

Abstract—The Transmission Control Protocol (TCP) underpins a large fraction of Internet communication, making unintended information leakage in its implementations a critical security concern. In particular, off-path attackers can exploit side-channel vulnerabilities to infer sensitive information without directly observing traffic. Such vulnerabilities can be analyzed by proving non-interference—a hyperproperty stating that secret inputs do not influence publicly observable outputs. Traditional formal verification methods, such as model checking and symbolic execution, suffer from state space explosion when applied to monolithic protocol implementations. This paper presents a lightweight, developer-centric verification framework that decomposes TCP implementations into small, unit-test-style program slices, each targeting a specific class of side-channel leakage. Each slice is analyzed using self-composition and bounded model checking with CBMC. The framework is evaluated across three TCP slices derived from known vulnerability classes, including CVE-2016-5696 and IPID-based inference attacks. The results show that the approach successfully identifies non-interference violations in all vulnerable models and verifies patched versions as secure under bounded executions. In addition, the framework produces precise, actionable counterexamples and is computationally efficient, with verification times ranging from milliseconds to minutes depending on the execution bound. The results demonstrate that slicing enables scalable and interpretable verification of TCP side-channel behavior, significantly reducing the complexity of full-system analysis while preserving the ability to detect meaningful security violations.

Index Terms—Non-interference property, side-channel attacks, automatic vulnerability detection

I. INTRODUCTION

This study seeks to detect violations of the non-interference property in localized TCP models [1]. Such violations can result in unintended information leakage, facilitating side-channel attacks such as TCP session hijacking [2]. Prior measurements indicate that a significant portion of Internet traffic depends on TCP-based protocols, so weaknesses in its behavior may have widespread consequences [3].

In practical implementations, such as the Linux kernel networking stack, multiple TCP connections share system resources, including buffers, timers, and connection tables. Insufficient isolation of these resources can cause the state of one connection to indirectly affect the behavior of another. Over the past decade, researchers have shown that such interference can create TCP side-channels. These channels enable attackers to infer sensitive information about victim connections, potentially facilitating attacks or inference of host characteristics without disclosing the attacker’s identity.

The non-interference property offers a formal framework for assessing the absence of information leakage [4]. Informally, it

stipulates that secret or high-security inputs must not influence publicly observable outputs. Since side-channels result from unintended information flows, verifying non-interference provides a formal method for detecting potential side-channel vulnerabilities. However, this verification is challenging because non-interference is a hyperproperty, requiring analysis of relationships across multiple system executions rather than a single execution trace. Consequently, specialized techniques such as model checking, symbolic execution, and self-composition are typically necessary.

While prior research has successfully employed model checking to expose TCP side-channels, existing methodologies frequently rely on manually constructed formal models requiring substantial domain expertise [5]. This reliance creates a bottleneck for both scalability and reproducibility. Automated frameworks, such as SCENT and SCAD, have demonstrated the feasibility of systematic detection; however, these systems depend on heavyweight symbolic execution engines and massive hardware profiles [6, 7]. The extreme computational cost and complex modeling requirements of these tools often render them inaccessible for rapid, iterative analysis. Consequently, there remains a critical need for a developer-centric verification approach that maintains formal rigor without the prohibitive overhead of whole-stack symbolic execution.

To address these limitations, this study proposes a practical approach to detecting potential side-channel leaks by decomposing TCP into a set of smaller, verifiable models, each targeting a specific interference mechanism, including execution timing, challenge ACK rate limiting, and IPID leakage [8, 9]. The approach utilizes the CBMC tool for automated analysis and applies self-composition to detect information leakage through unit-test-style components. This method improves scalability and usability while preserving the essential behaviors that cause side-channel leakage, thereby enabling more accessible verification. To further ensure tractability, the method adjusts the exploration space through bounded analysis, achieving a practical balance between rigor and performance.

However, this proposed approach is insufficient to establish global non-interference in TCP implementations due to limitations in slicing, abstraction, and bounded exploration. It may not detect interference caused by cross-component interactions, extended execution traces, or omitted protocol mechanisms—leading to false negatives. However, many documented TCP side-channels arise from localized shared-state interactions, including global counters, which are effectively

captured by the slicing methodology. Therefore, although this approach cannot guarantee the absence of interference across the entire system, it is effective at identifying a significant class of vulnerabilities while substantially reducing verification costs.

This approach also relies on developer-guided abstraction to construct minimal program slices that encompass all relevant shared resources and potential interference channels. This reliance introduces a modeling assumption: verification completeness depends on the adequacy of the selected abstraction boundary. Within a correctly specified slice, CBMC produces sound counterexamples; however, violations do not extend beyond the modeled components.

Nevertheless, the proposed approach provides a structured starting point that can be extended and scaled to identify potential side-channels in TCP logic. It enables localized analysis with relatively low engineering effort and fast execution times (seconds to minutes), while offering targeted, deep coverage and actionable debugging.

II. STATE OF THE ART

Manual Detection: Initial investigations into TCP-side channels primarily relied on manual analysis of protocol implementations. Researchers analyzed widely deployed networking stacks to identify behaviors that could leak information about other connections. These studies revealed that attackers could deduce TCP connection states, sequence numbers, and port numbers by transmitting specially crafted probe packets and monitoring server responses [2, 10]. Although these studies identified significant vulnerabilities, the discovery process relied heavily on expert knowledge of TCP internals and detailed examination of specific implementations. Consequently, manual analysis is not scalable due to the complexity and diversity of contemporary networking stack implementations.

Automatic Detection: In contrast to manual discovery, recent research aims to automatically identify potential information leaks using formal verification methods [5]. Model checking, in particular, has been used to analyze network protocols and identify subtle interactions that may lead to vulnerabilities [11]. These techniques systematically explore protocol states and execution paths to verify whether specified properties are maintained. However, applying model checking directly to a complete TCP implementation is challenging due to the protocol’s extensive state space and the complexity of its implementation [5, 12]. Exhaustive verification of all possible states is generally infeasible without significant abstraction.

The SCENT framework introduces an automated downscaling method that transforms TCP code into a form suitable for model checking by reducing the state space and limiting the number of inputs considered during verification [6]. Another system, SCAD, seeks to detect violations of the non-interference property through symbolic execution, identifying potential information flows between connections [7].

Despite these advancements, significant barriers to practical adoption remain. SCENT requires a high degree of manual

protocol abstraction to remain tractable. In contrast, SCAD’s reliance on symbolic execution leads to a “path explosion” problem that is only partially mitigated by strict execution bounds. Both frameworks demand specialized domain expertise to configure and require substantial manual effort to construct representative models. Furthermore, the operational overhead is significant: previous assessments indicate that these tools require several hours of runtime and consume hundreds of gigabytes of memory during large-scale analyses. Additionally, their development cycles often extend over days or weeks. These limitations motivate a lighter-weight, modular approach that decomposes TCP into independently verifiable slices, enabling targeted analysis without whole-system modeling.

III. BACKGROUND

TCP is a connection-oriented transport-layer protocol that ensures reliable, ordered, and error-checked delivery of data streams between applications on IP networks. Reliability is maintained through sequence numbers and acknowledgments, and connections are established via a three-way handshake consisting of SYN, SYN-ACK, and ACK packets, as specified in RFC 9293 [1]. A tuple of source and destination IP addresses and port numbers uniquely identifies each TCP connection. While certain ports are reserved for well-known services, such as ports 80 and 443 for HTTP and HTTPS, client-side ephemeral ports are generally unpredictable.

If an attacker can infer information such as port numbers or sequence numbers, multiple attacks become feasible. For instance, an attacker could determine whether two hosts are communicating [13], estimate the volume of data exchanged [14], terminate an active connection by sending a reset (RST) packet [15], or hijack a session by injecting forged packets [8, 16]. These risks are especially pronounced when communication occurs over unencrypted protocols, such as HTTP over TCP. However, even over encrypted channels such as HTTPS, TCP-level side-channels remain a concern: attackers can infer sensitive information from observable metadata, including packet timing, sizes, and transmission patterns [17, 18]. These signals may still leak information about connection state or activity despite encryption.

In systems where inputs and outputs are classified by security level, such as public and secret, the non-interference property requires that high-security inputs do not influence low-security outputs. Because side-channels arise from unintended information flows, non-interference is widely used as a formal condition for verifying the absence of information leakage. A common approach to modeling non-interference is with self-composition, in which a program is executed in parallel with itself with different secret inputs, enabling the verifier to determine whether the observable outputs remain equivalent [19].

Side-channel attacks exploit indirect signals that disclose information about a system’s internal state. Rather than accessing protected data directly, attackers infer secrets by observing

system behaviors such as timing, resource usage, or communication patterns [20]. In networked systems, when shared resources are present, the responses to an attacker’s probes can become correlated with the activity of other connections. If these responses vary based on the hidden state of a victim connection, an attacker may infer sensitive information without directly observing the victim’s traffic. From an information-flow security perspective, such behavior violates the non-interference property.

IV. THREAT MODEL

This study examines side-channel attacks in which an adversary seeks to infer sensitive information about an active TCP connection. The system model comprises three entities: a victim client, a victim server, and an off-path attacker. The client communicates with the server via TCP, while the attacker sends probe packets and observes responses. In some cases, spoofed packets may be sent to the client, but the primary observation channel in this research is server responses. The model abstracts away network-level details such as routing paths, number of hops, and shared physical infrastructure.

Shared resources within the network stack allow the behavior of one connection to influence the observable responses of another. By probing these interactions, an attacker may infer hidden state information, such as the client’s ephemeral port, the TCP sequence number expected by the server, and the acknowledgment number.

The attacker is assumed to be able to send and receive packets over the Internet and may spoof IP addresses. Attacks that require physical access, compromise of the victim client or server, or privileged access to internal state are considered out of scope. Attacker observations include packet responses such as ACKs or the absence of a response, as well as metadata such as timing, depending on the modeled slice.

V. DESIGN

TCP side-channel behavior is modeled using a simplified abstraction of a TCP server that manages two connections: a victim connection containing high-security (secret) inputs and an attacker connection observing low-security outputs. High-security inputs consist of the victim’s ephemeral port number and TCP sequence number, while low-security outputs comprise server response packets and their associated metadata. Both connections share internal resources—including execution timing behavior, global counters (such as Challenge ACK rate limiting), and IPID assignment structures—which may create side-channels. Within the models, a vulnerability is defined as a violation of non-interference, where variations in secret inputs produce observable differences in attacker-visible outputs.

Simplified TCP Implementation: To facilitate manageable verification, the analysis focuses on carefully chosen slices of the TCP implementation. Each TCP slice is treated as a unit test, with the non-interference property serving as the test oracle. These slices preserve the essential protocol behaviors

needed to reveal vulnerabilities and evaluate patches, while significantly reducing the verification state space.

The models include core TCP connection-management logic, such as connection states (e.g., established flags), sequence-number validation, Challenge ACK handling, and shared counters or state variables. The TCP logic is implemented in C to process incoming packets, update each connection’s state, and verify behavior using CBMC. To further reduce complexity, certain features commonly found in production TCP stacks (such as congestion control algorithms, retransmission mechanisms, and advanced protocol options) have been omitted.

Non-Interference Verification: Because non-interference is a hyperproperty that relates multiple executions of a system, self-composition is used. During the first execution, the victim connection uses a secret value (*secret1*). In the second execution, the victim connection uses a different secret value (*secret2*). All attacker inputs remain identical across both executions. The verifier then examines, within the explored bound, whether attacker-visible outputs remain identical across both executions for all possible behaviors, thus establishing non-interference. Formally, the non-interference condition is expressed as:

$$\begin{aligned} \forall \text{secret}_1, \text{secret}_2, \text{attackerInput}, \\ \text{obs}(\text{Prog}(\text{secret}_1, \text{attackerInput})) \equiv \quad (1) \\ \text{obs}(\text{Prog}(\text{secret}_2, \text{attackerInput})) \end{aligned}$$

If observable outputs differ, the attacker can distinguish between secret states, indicating a potential information leak.

Figure 2 presents a simplified timing-based example in which the server processes an incoming packet and incurs additional delay when the attacker’s guessed port matches the secret port. If the secret port is 80 and the attacker guesses correctly, the observed execution time may be higher than when the secret port is 8080, and the guess is incorrect. Because the observed timing differs across executions with different secrets, the equality condition fails, demonstrating a violation of non-interference.

VI. IMPLEMENTATION

Three distinct TCP side-channel models were developed, each representing a specific vulnerability and its corresponding patch: (1) a timing-based port inference model, (2) a Challenge ACK rate-limiting model based on CVE-2016-5696, and (3) an IPID downgrade-based information leakage model. The latter two models were constructed following a comprehensive review of the relevant literature [8, 9].

The port inference model assesses a timing side-channel vulnerability in which execution time varies depending on whether an attacker’s guessed port matches the secret port. This scenario is modeled in a simplified setting that assumes a perfectly deterministic environment with no external variability. It also assumes that the off-path attacker has already identified the target’s IP address. The model evaluates whether the attacker can infer the sensitive TCP port number—the

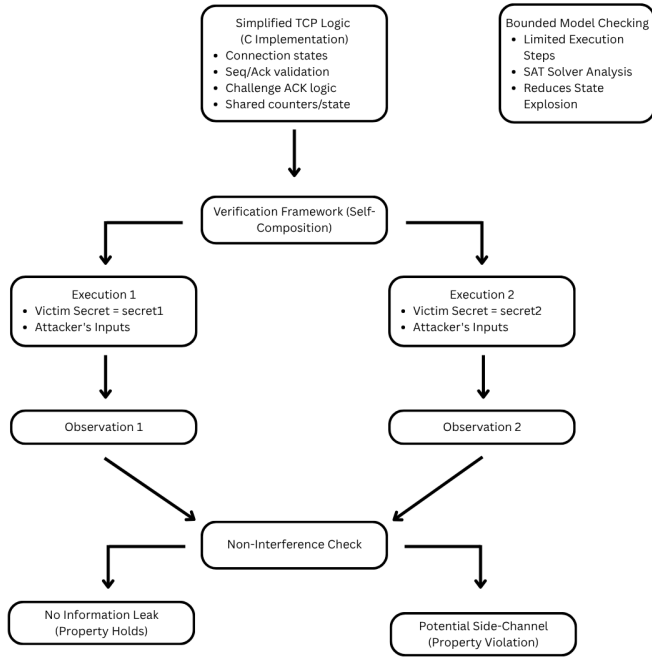


Fig. 1. Framework Pipeline

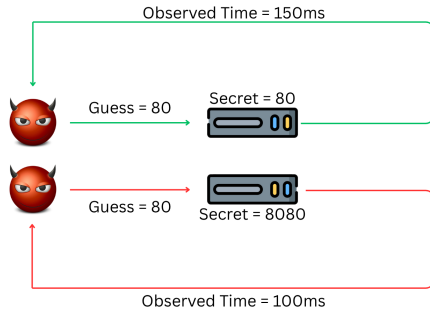


Fig. 2. Timing-Based TCP Side-Channel Model (Port Inference)

remaining component of the connection’s four-tuple—by observing data-dependent execution times, thereby exploiting a timing side-channel. The model simulates packet processing, where correct guesses introduce additional delay and incorrect guesses yield faster responses. Using self-composition, the model executes two traces with different secret ports and asserts that the observed execution times must be identical. Any violation demonstrates that timing differences disclose information about the secret port. The corresponding patch enforces constant-time behavior by ensuring that both matching and non-matching cases incur identical delays, thereby eliminating timing-based leakage and restoring non-interference within the analyzed slice.

The Challenge ACK rate-limiting model captures a side-channel arising from a shared global counter that limits the number of Challenge ACK responses. This model assumes

```
#define PORT_MIN_NUMBER 0
#define PORT_MAX_NUMBER UINT16_MAX // The maximum port number for TCP is 65,535

int nondet_int(); // Allows exploration of all possible integer values under the given constraints
int elapsed_time; // Represents how long a model takes to execute (in milliseconds)

void reset() { elapsed_time = 0; } // Restores independence between the two executions

typedef struct {
    int port; // Secret for victim
    bool active;
} tcp_conn;

void process_packet(tcp_conn *conn, int probed_port, int extra_delay) {
    if (conn->active) // Basic overhead for every packet
        elapsed_time += 10;

    // Correct port probes trigger additional processing (e.g., connection lookup)
    if (conn->active && probed_port == conn->port)
        elapsed_time += delay; // secret-dependent timing
}

int run_trace_timing(int secret_port, int attacker_port_guess, int delay) {
    reset();
    tcp_conn victim = { .port = secret_port, .active = true };
    process_packet(&victim, attacker_port_guess, delay); // Attacker sends a probe
    return elapsed_time;
}

int main() {
    int secret_port_1 = nondet_int(), secret_port_2 = nondet_int();
    int attacker_port_guess = nondet_int();
    int delay = nondet_int(); // Defines the delay once, so it is the same for both runs

    __CPROVER_assume(PORT_MIN <= secret_port_1 && secret_port_1 <= PORT_MAX);
    __CPROVER_assume(PORT_MIN <= secret_port_2 && secret_port_2 <= PORT_MAX);
    __CPROVER_assume(secret_port_1 != secret_port_2);

    __CPROVER_assume(PORT_MIN <= guess && guess <= PORT_MAX);
    __CPROVER_assume(100 <= delay && delay <= 500);

    int time_1 = run_trace_timing(secret_port_1, attacker_port_guess, delay);
    int time_2 = run_trace_timing(secret_port_2, attacker_port_guess, delay);

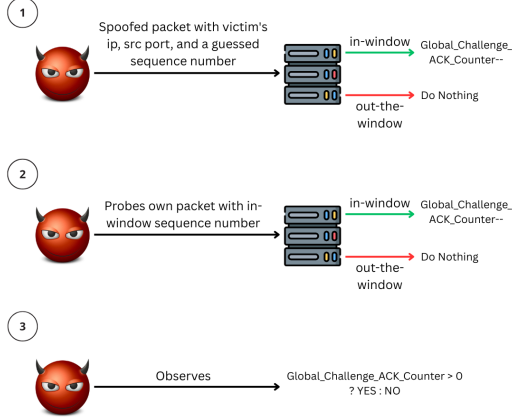
    assert(time_1 == time_2); // Non-interference (timing indistinguishability)
}
```

Fig. 3. Port Inference Vulnerability Model

that the off-path attacker has already determined the correct IP address and port number for the victim’s connection, for example, through a prior port-probing side-channel. The objective is to evaluate whether the attacker can subsequently infer the sensitive TCP sequence number. Within this model, the attacker transmits sequence-number guesses to the victim’s connection and subsequently probes their own connection to assess whether the global counter has been depleted. If the attacker’s observations vary based on whether the guesses fall within the victim’s receive window, the attacker can deduce the victim’s secret sequence number. The patched implementation addresses this side-channel by replacing the global counter with a per-connection counter, thereby isolating each connection’s behavior and preventing information leakage via shared state within the modeled slice.

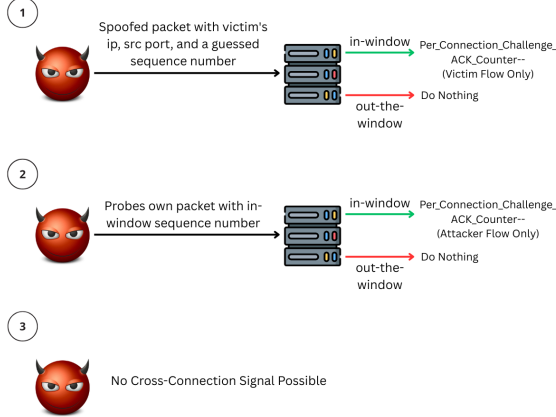
The IPID downgrade model describes a side-channel attack in which an adversary induces the system to transition from a secure per-socket IPID assignment policy to a less secure hash-based global counter. By detecting a hash collision between the adversary’s and the victim’s connections, the attacker can monitor changes in the shared counter while transmitting in-window sequence-number guesses. This vulnerability enables the attacker to infer both the existence and the state of the victim’s connection. The patched implementation mitigates this downgrade by ensuring that TCP packets consistently use

Global Rate Limit



(a) Attacker Trace under Global Rate Limiting

Per-Connection Rate Limit



(b) Attacker Trace under Per-Connection Rate Limiting

Fig. 4. Resource-Contention TCP Side-Channel Model (CVE-2016-5696)

per-socket IPID counters, while shared counters are limited to non-TCP traffic. This modification removes the shared resource dependency and restores non-interference within the slice.

VII. EVALUATION

This evaluation investigates three research questions that assess the correctness, scalability, and practical utility of the proposed unit-test-style slicing framework for verifying side-channel vulnerabilities.

Research Question 1: Can the framework distinguish between vulnerable and secure TCP implementations?

Correctness is evaluated by inspecting whether the model checker detects violations of the non-interference property in vulnerable models and verifies its preservation in patched

TABLE I
VULNERABILITY DETECTION AND PATCH VERIFICATION RESULTS

Model Slice	Code Size (Vulnerability / Patch)	Vulnerability Detected	Patch Verified (Bounded)	Counterexample Observation
Port Inference	90 / 95 LOC	Yes ($K \geq 1$)	Yes	Time: 122 ms vs. 10 ms
CVE-2016-5696	139 / 126 LOC	Yes ($K \geq 2$)	Yes	CHALLENGE ACK vs. NO RESPONSE
PID downgrade	169 / 180 LOC	Yes ($K \geq 1$)	Yes	ID INCREMENTED vs. ID STABLE

counterparts within the explored bounds. A successful outcome requires (1) the generation of concrete counterexamples for vulnerable implementations and (2) successful verification (no assertion violations) for patched versions. Across the three case studies—port inference, CVE-2016-5696, and IPID downgrade—the framework consistently satisfies this criterion within the explored bounds. All vulnerable models produced counterexamples showing observable differences in attacker outputs for identical attacker inputs, whereas all patched models exhibited no non-interference violations across all tested bounds for $K \in \{1, 2, 3, 5, 10\}$. Here, K represents the maximum number of scheduler steps (e.g., context switches, interleavings, or attacker actions) explored during the self-composed execution. Table I summarizes the results.

Beyond simply detecting violations, the framework produces counterexamples that are directly interpretable and actionable.

- In the port inference model, identical attacker inputs yield significantly different execution times (e.g., 122 ms versus 10 ms), enabling the attacker to infer the secret port with high confidence. The patched model eliminates this leak by enforcing constant-time behavior.
- In the CVE-2016-5696 model, leakage arises from contention on a shared Challenge ACK counter. When the attacker's guess falls within the TCP receive window in one execution but not the other, the shared resource is consumed asymmetrically, leading to distinguishable outcomes (CHALLENGE ACK versus NO RESPONSE) when the attacker probes their own connection. The patched version removes this channel by introducing per-connection rate limiting, thereby eliminating shared state.
- Similarly, in the IPID downgrade model, the attacker exploits a shared global counter by forcing a downgrade in IPID assignment, then observing whether the counter increments to determine whether a guessed sequence number is valid. The patch prevents this by enforcing protocol-based isolation, ensuring that TCP traffic cannot influence shared counters.

These results also demonstrate that the framework detects both single-step and multi-step side-channel behaviors: the port inference model exhibits violations at ($K = 1$), whereas the CVE-2016-5696 model requires ($K \geq 2$) to reveal interference.

Research Question 2: How does increasing the execution bound (K) affect performance?

To evaluate scalability, runtime and memory usage are measured as the execution bound (K) increases. Increasing

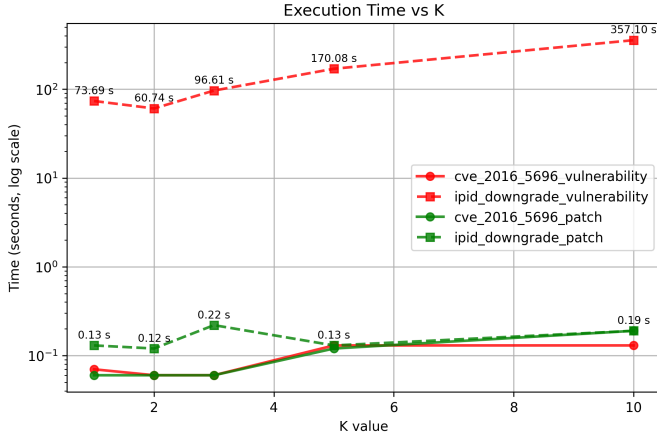


Fig. 5. Execution Time vs. K

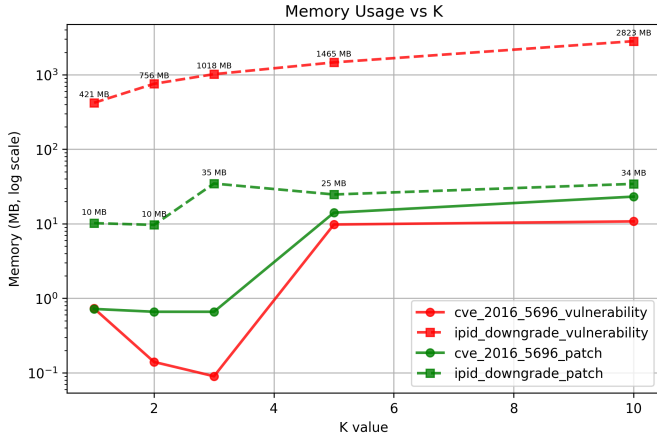


Fig. 6. Memory Usage vs. K

K expands the state space by allowing longer interleavings and deeper execution traces, thereby increasing both runtime and memory usage. For CVE-2016-5696, verification remains highly efficient even at $K = 10$, with runtimes under a fraction of a second and minimal memory usage. In contrast, the IPID downgrade vulnerability model incurs significantly higher computational cost due to the inclusion of a large array of hash-indexed counters (2048 entries) and repeated interactions with the attacker. As K increases, verification time grows to several minutes (up to 357 seconds), and memory usage approaches several gigabytes (up to ~ 2.8 GB). Nevertheless, verification remains tractable within minutes rather than hours or days, indicating that the slicing approach substantially mitigates complexity while still enabling the detection of multi-step side-channel behaviors.

Research Question 3: How does the proposed unit-test-style slicing method compare to the whole-stack modeling approaches?

The proposed unit-test-style slicing methodology is now compared to prior whole-stack modeling approaches. Tradi-

TABLE II
COMPARATIVE ANALYSIS OF UNIT-TEST-STYLE SLICING AND WHOLE-STACK MODELING

Approach	Modeling Effort	Memory Use	Runtime	Actionality
SCENT	~ 2.5 person-weeks	Up to 727 GB of memory	~ 3.47 hours (12,492 seconds)	High: 12 newly identified vulnerabilities
SCAD	~ 1 person-day (labeling only)	Up to 2 TB of memory	3,089 CPU hours	High: 7 newly identified
Unit-Test Slices	1 person, a few hours	< 3 GB memory	< 6 minutes	High: precise counterexamples identified

tional systems such as SCENT and SCAD require substantial engineering effort to construct full-system models, often taking days or weeks of development time and necessitating specialized hardware with hundreds of gigabytes, or even terabytes, of memory [6, 7]. In contrast, the models in this work are compact, each consisting of fewer than 200 lines of code, and can be developed by a single person in a matter of hours. Verification can be performed on commodity hardware, as demonstrated by experiments conducted on a 24 GB machine, with runtimes ranging from seconds to minutes. Despite this reduction in scale, the sliced models capture realistic side-channel behaviors and successfully reproduce known vulnerabilities with precise counterexamples within the explored bounds.

This comparison highlights a key tradeoff. Whole-stack approaches provide stronger end-to-end guarantees but at the cost of significant complexity, resource requirements, and development effort. In contrast, the proposed slicing approach sacrifices full-system coverage in favor of modularity, efficiency, and ease of use. The proposed approach enables rapid experimentation and debugging while still preserving the ability to reason rigorously about specific side-channel vulnerabilities.

It is important to note that this is not a direct “like-for-like” comparison. Systems like SCENT utilize full TCP implementations, whereas the models in this work are custom-built and simplified.

VIII. CONCLUSION

A lightweight verification framework was introduced for detecting non-interference violations in TCP state machine slices. By decomposing TCP into unit-test-style components and applying self-composition with bounded model checking, this approach enables efficient detection of side-channel vulnerabilities without requiring full protocol modeling. Across three representative case studies, the framework successfully identified known vulnerabilities by generating precise, actionable counterexamples and detected no violations in the corresponding patched models within the explored bounds.

The results indicate that slicing offers a practical balance between scalability and coverage. Although it does not guarantee global non-interference, it enables rapid and developer-friendly analysis of localized interference patterns. Future research may explore the use of large language models (LLMs) to extend slice composition to capture cross-component interactions and automate slice extraction from real-world TCP implementations.

REFERENCES

- [1] W. Eddy, “Transmission Control Protocol (TCP),” RFC 9293, Internet Engineering Task Force (IETF), Aug. 2022. [Online]. Available: <https://datatracker.ietf.org/doc/rfc9293/>
- [2] Z. Wang et al., “Off-Path TCP Hijacking in Wi-Fi Networks: A Packet-Size Side Channel Attack,” in Proc. Network and Distributed System Security Symposium (NDSS), Internet Society, 2025. [Online]. Available: <https://www.ndss-symposium.org/wp-content/uploads/2025-305-paper.pdf>
- [3] L. O. Ostrowsky, N. L. S. da Fonseca, and C. A. V. Melo, “A multiscaling traffic model for UDP streams,” *Simulation Modeling Practice and Theory*, vol. 26, pp. 32–48, May 2012, doi: 10.1016/j.simpat.2012.04.003.
- [4] J. A. Goguen and J. Meseguer, “Security Policies and Security Models,” in Proc. IEEE Symp. Security and Privacy, Oakland, CA, USA, 1982, pp. 11–20, doi: 10.1109/SP.1982.10014.
- [5] M. Musuvathi and D. Engler, “Model Checking Large Network Protocol Implementations,” Mar. 2004. Accessed: Apr. 08, 2026. [Online]. Available: <https://web.stanford.edu/~engler/nsdi04.pdf>
- [6] Y. Cao, Z. Wang, Z. Qian, C. Song, S. V. Krishnamurthy, and P. Yu, “Principled Unearthing of TCP Side Channel Vulnerabilities,” in Proc. ACM SIGSAC Conf. Computer and Communications Security, 2019, pp. 211–224, Nov. 2019, doi: 10.1145/3319535.3354250.
- [7] K. Man et al., “SCAD: Towards a Universal and Automated Network Side-Channel Vulnerability Detection,” in IEEE Symp. Security and Privacy, pp. 1861–1876, 2025, doi: 10.1109/SP61157.2025.00068.
- [8] Y. Cao et al., “Off-Path TCP Exploits: Global Rate Limit Considered Dangerous,” in Proc. USENIX Security Symp., 2016, pp. 209–225. [Online]. Available: <https://www.usenix.org/system/files/conference/usenixsecurity16/sec16-paper-cao.pdf>
- [9] X. Feng et al., “Off-Path TCP Hijacking Attacks via the Side Channel of Downgraded IPID,” *IEEE/ACM Trans. Netw.*, vol. 30, no. 1, pp. 409–422, Feb. 2022, doi: 10.1109/TNET.2021.3115517.
- [10] Z. Qian, Z. M. Mao, and Y. Xie, “Collaborative TCP sequence number inference attack: how to crack sequence number under a second,” in Proc. ACM CCS, 2012, pp. 593–604, doi: 10.1145/2382196.2382258.
- [11] G. Cluzel et al., “Layered Formal Verification of a TCP Stack,” in IEEE Secure Development Conf. (SecDev), 2021, pp. 86–93, doi: 10.1109/SECDEV51306.2021.00028.
- [12] Y. Li et al., “A Survey on Network Verification and Testing With Formal Methods: Approaches and Challenges,” *IEEE Commun. Surveys & Tutorials*, vol. 21, no. 1, pp. 940–969, 2019, doi: 10.1109/COMST.2018.2868050.
- [13] R. Ensafi et al., “Idle port scanning and non-interference analysis of network protocol stacks using model checking,” in Proc. USENIX Security Symp., 2010. [Online]. Available: https://www.usenix.org/legacy/events/sec10/tech/full_papers/Ensafi.pdf
- [14] W. Chen and Z. Qian, “Off-Path TCP Exploit: How Wireless Routers Can Jeopardize Your Secrets,” in Proc. USENIX Security Symp., 2018. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity18/presentation/chen-weiteng>
- [15] C. L. Schuba et al., “Analysis of a denial of service attack on TCP,” in Proc. IEEE Symp. Security and Privacy, 1997, pp. 208–223, doi: 10.1109/SECPRI.1997.601338.
- [16] X. Feng et al., “Off-Path TCP Exploits: PMTUD Breaks TCP Connection Isolation in IP Address Sharing Scenarios,” arXiv:2509.11833, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2509.11833>
- [17] A. Shusterman et al., “Robust Website Fingerprinting Through the Cache Occupancy Channel and its Real World Practicality,” *IEEE Trans. Dependable Secure Comput.*, 2020, doi: 10.1109/TDSC.2020.2988369.
- [18] A. Shusterman et al., “Prime+Probe 1, JavaScript 0: Overcoming Browser-based Side-Channel Defenses,” 2021. [Online]. Available: <https://www.usenix.org/system/files/sec21-shusterman.pdf>
- [19] G. Barthe, P. R. D’Argenio, and T. Rezk, “Secure information flow by self-composition,” *Math. Struct. Comput. Sci.*, vol. 21, pp. 1207–1252, 2011, doi: 10.1017/S0960129511000193.
- [20] National Security Agency, “TEMPEST: A Signal Problem,” 1972. [Online]. Available: <https://www.nsa.gov/portals/75/documents/news-features/declassified-documents/cryptologic-spectrum/tempest.pdf>